P1667R0
Concept-aware `noexcept` specifiers

Published Proposal, 2019-06-17

This version:
https://wg21.link/p1725

Author:
Christopher Di Bella

Audience:
SG17

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This proposal describes an extension to the core language that will improve the maintenance of conditional `noexcept` specifications so that they have synergy with the interface required by concepts.

Table of Contents

§ 1. Motivation

Conditional `noexcept` specifications can be easy to write.

```
template<Movable T>
T example1(T&&) noexcept;

template<Movable T>
T example2(T&&) noexcept(is_nothrow_move_constructible_v<T> and is_nothrow_move_assignable_
```

They can also be painful to write, and thus painful to read. Consider writing a `noexcept` specifier for `std::ranges::find`:

```
template<InputIterator I, Sentinel<I> S, class T, class Proj = identity>
requires IndirectRelation<I, T*, ranges::equal_to, Proj>
I find(I first, S last, const T& val, Proj proj = {})
   noexcept(
      is_nothrow_move_constructible_v<I> and is_nothrow_move_assignable_v<I> and
      is_nothrow_copy_constructible_v<I> and is_nothrow_copy_assignable_v<I> and
      is_nothrow_destructible_v<I> and
      is_nothrow_move_constructible_v<S> and is_nothrow_move_assignable_v<S> and
      is_nothrow_copy_constructible_v<S> and is_nothrow_copy_assignable_v<S> and
      is_nothrow_destructible_v<S> and
      noexcept(first != last) and
      noexcept(++first) and
      noexcept(*first)
      noexcept(*first == val)
   )
{
   for (; first != last; ++first) {
      if (*first == val) {
         break;
      }
   }
   return first;
}
```

The above `noexcept`-specifier should capture all the expressions in the algorithm (although it is entirely possible that the author "honestly" missed something, which should serve as further motivation for this proposal).

That's a lot to read, and since iterators are broadly used in the standard library, it might make more sense for us to create `constexpr` template variables à la `is_nothrow_<Concept>`, but these will largely be repeating the definition of a concept. Take `is_nothrow_semiregular` for example:

```
template<Semiregular T>
constexpr is_nothrow_semiregular = is_nothrow_default_constructible<T> and is_nothrow_copya
```

We're now beholden to defining `is_nothrow_default_constructible` and `is_nothrow_copyable`, and we can't define `is_nothrow_default_constructible` as `std::is_nothrow_default_constructible_v`, since it would not check that the type's destructor is non-throwing. Similarly, `is_nothrow_copyable` would need to be defined such that it checks `T`'s copy constructor, move constructor, and respective assignment operators don't throw, in addition to the destructor (if something is truly no-throw copyable, it should most probably have a non-throwing destructor too). At this point, it looks as though we might be considering redefining all the concepts introduced in C++20 for `noexcept`-specifiers.

```
template<InputIterator I, Sentinel<I> S, class T, class Proj = identity>
requires IndirectRelation<I, T*, ranges::equal_to, Proj>
constexpr I find(I first, S last, T const& value, Proj proj)
   noexcept(is_nothrow_input_iterator<I> and is_nothrow_sentinel<S, I> and
            is_nothrow_indirect_relation<I, T*, ranges::equal_to, Proj>);
```

This is most certainly repeating the interface unnecessarily: we have all of the information we need in the form of concepts. PXXXX seeks to introduce `noexcept(requires)` -- or some other spelling —that checks all requirements within a *requires-expression*: that is, `noexcept(requires)` is equivalent to `noexcept(true)` if, and only if, all expressions in all (transitive) requirements are `noexcept(true)` also.

```
// Example 1
class nothrow_example {
public:
   nothrow_example() = default;

   constexpr explicit nothrow_example(int const value) noexcept
      : value_{value}
   {}

   constexpr int size() const noexcept
   { return value_; }
private:
   int value_ = 0;
};

template<Semiregular T>
auto possibly_nothrow(T t) noexcept(requires)
{
   return t.size() * 2;
}

int main()
{
   possibly_nothrow(nothrow_example{42}); // is noexcept
   possibly_nothrow(vector<int>(1'000'000)); // is not noexcept
}
```

In *Example 1*, `possibly_nothrow<nothrow_example>` is deemed to be a `noexcept` function because none of its operations potentially throw. `possibly_nothrow<vector<int>>` is potentially throwing due to its copy operations being potentially throwing.

```
// Example 2
class mostly_nothrow {
public:
   mostly_nothrow() = default;

   constexpr explicit mostly_nothrow(int const value)
      : value_{value}
   {}

   constexpr int size() const
   { return value_; }
private:
   int value_ = 0;
};

// ...
possibly_nothrow(mostly_nothrow{42});
```

In *Example 2*, `possibly_nothrow<mostly_nothrow>` is *also* a `noexcept` function, because the interface specified through the defined concepts do not check for `Constructible<int>`. Just as we would get a template instantiation error if we tried to call `possibly_nothrow(0)` due to `int::size` not existing, `possibly_nothrow<mostly_nothrow>` slips through the cracks because its potentially-throwing constructor is not checked in any way.

```
// Example 3
class actually_throws {
public:
   actually_throws() = default;

   constexpr explicit actually_throws(int const value) noexcept
      : value_{value}
   {}

   constexpr void size() const
   { throw value_; }
private:
   int value_ = 0;
};

// ...
possibly_nothrow(actually_throws{42});
```

In *Example 3*, `possibly_nothrow<actually_throws>` is also `noexcept` for the same reasons as in *Example 2*.

```
// Example 4
template<class T>
auto possibly_nothrow(T const& t) noexcept(requires)
{
   return t.size();
}
```

Example 4 could either be equivalent to `noexcept(true)` or it could be ill-formed, as this is a completely unconstrained overload of `possibly_nothrow`. The author has no strong opinion on which direction to take.

§ 1.1. Before-and-after tables

C++20	

`template<InputIterator I, Sentinel<I> S, class T, class Proj = identity>`
`requires IndirectRelation<I, T*, ranges::equal_to, Proj>`
`I find(I first, S last, const T&amp; val, Proj proj = {})`
`  noexcept(`
`  is_nothrow_move_constructible_v<I> and`
`  is_nothrow_move_assignable_v<I> and`
`  is_nothrow_copy_constructible_v<I> and`
`  is_nothrow_copy_assignable_v<I> and`
`  is_nothrow_destructible_v<I> and`
`  is_nothrow_move_constructible_v<S> and`
`  is_nothrow_move_assignable_v<S> and`
`  is_nothrow_copy_constructible_v<S> and`
`  is_nothrow_copy_assignable_v<S> and`
`  is_nothrow_destructible_v<S> and`
`  noexcept(first != last) and`
`  noexcept(++first) and`
`  noexcept(*first)`
`  noexcept(*first == val)`
`  );`
	`template<InputIterator> I, S`
	`requires IndirectRelation<I,`
	`I find(I first, S last, cons`

2. Presumed criticisms

2.1. This is a rebranding of `noexcept(auto)`

Through online chatter, the author is of the opinion that `noexcept(auto)` has become a loaded term to mean something along the lines of "check all expressions in a function definition and only if all of them are `noexcept`, then this function is `noexcept` also". `noexcept(requires)` only focuses on the expressions required by the function template declaraiton.

2.2. Are you serious about the spelling!?

This proposal makes no intention to word-smith. `noexcept(requires)` was chosen because it couples the idea of `noexcept` and *requires-expression*s for communication with EWG. The author encourages bikeshedding a different spelling, but strongly adivses against spelling this language feature as `noexcept(auto)`.

2.3. If you're confident that this *isn't* `noexcept(auto)`, then it's broken, because...

> ... a concept might contain features that a function doesn't use. e.g.
>
> ```
> // Let SequenceContainer roughly represent the named requirement SequenceContainer from
> // [container.requirements.general] Table 62
> //
> void clear(SequenceContainer auto& c) noexcept(requires)
> {
>    c.clear();
> }
> ```

Java provides the language feature `interface` to dictate what the interface of a given class looks like. For those unfamiliar with Java, an `interface` prescribes a type's interface, dictating the minimum number of member functions that a class conforming to the interface must provide; otherwise it is an abstract class. The closest that C++ has to offer here are virtual functions.

It is not the primary purpose of concepts to mimic Java's `interface`: it is to expres requirements on type parameters through concepts is to describe the *minimal* interface needed for well-formed usage. This is why the range-based `std::ranges::find` requires an `InputRange`, not a `Container`. The fact that concepts *can* be used similarly to Java's `interface` is a happy coincidence. The author suspects that programmers using concepts as a mechanism for `interface` is likely inevitable, but the C++ spelling for `class vector<T> implements SequenceContainer<T>` is `static_assert(SequenceContainer<vector<T>>);`, not `void clear(SequenceContainer auto& c);`. As such, the author deems that the `clear` function provided does not fall within the design-space of concept usage, and is not a valid counter-example.

> ... `InputIterator` refines `Copyable` and the copy happens outside the function call.

[P1207] has been forwarded to LWG for review.

3. Acknowledgements

The author would like to thank Isabella Muerte and Morris Hafner for their feedback on this proposal.

§ References

§ Informative References

[P1207]
Corentin Jabot. Movability of Single-pass Iterators. URL: https://wg21.link/p1207